Retarded Time via a Light-Cone Root Solve on the GPU

Newton iteration on eq. (1.2) vs the fixed-step RK4 march

Experiment `worktree-newton-lightcone` · AMD W7900 · July 2026

Summary. Replacing the per-pixel fixed-step RK4 march of the retarded-time ODE with a per-sample Newton solve of the light-cone condition makes the unified GPU Liénard–Wiechert kernels **faster at equal-or-better accuracy** on both code paths. At the production-like $a_0 = 0.1$ setup a **single** warm-started Newton correction per sample already sits at the accuracy floor (9.4×10^{-11} vs 9.5×10^{-10} for RK4 with `n_substeps = 1`), while running 1.5–2.0× faster on the potential path and time-neutral to 1.3× faster on the field path. Against the equal-accuracy RK4 configuration (`n_substeps = 8`) the speedup is 4–11×.

1 Problem

The Liénard–Wiechert accumulation evaluates, for every screen pixel $\boldsymbol{x}$ and every observer-time sample x^0_k, the trajectory state at the retarded proper time τ_r defined by the light-cone condition (eq. 1.2 of `FFT_v2.pdf`):

$$f(\tau) = x^0_k - x^0(\tau) - |\boldsymbol{x} - \boldsymbol{r}(\tau)| = 0. \tag{1}$$

The production kernels (`src/gpu/kernel_rk4.jl`) instead integrate the equivalent ODE (eq. 1.13)

$$\frac{\mathrm{d}\tau_r}{\mathrm{d}x^0} = \frac{1}{u^0 - \boldsymbol{u}\cdot\hat{\boldsymbol{n}}}, \quad \hat{\boldsymbol{n}} = \frac{\boldsymbol{x} - \boldsymbol{r}(\tau_r)}{|\boldsymbol{x} - \boldsymbol{r}(\tau_r)|}, \tag{2}$$

with classical RK4 at the sample resolution (`n_substeps` sub-steps per sample), one GPU thread per pixel. Each RK4 sub-step costs four evaluations of the trajectory spline (a `GPUCubicSpline` holding the 8-vector $[x^\mu; u^\mu](\tau)$), plus one more evaluation at the converged τ for the accumulation itself: $4n_{\mathrm{sub}} + 1$ spline evaluations per sample. Spline evaluations (binary knot search + cubic reconstruction) dominate the kernel cost, and the march **accumulates** integration error along the sample grid.

2 Method: warm-started Newton on the light-cone residual

Three structural facts make Equation 1 a better target than Equation 2 on the GPU:

1. **Monotonicity.** $f'(\tau) = -(u^0 - \boldsymbol{u}\cdot\hat{\boldsymbol{n}}) < 0$ strictly, since $u^0 \geq |\boldsymbol{u}|$ on a timelike worldline. The root is unique and Newton is well-behaved.
2. **The derivative is free.** $-1/f'$ is exactly the RHS of Equation 2, computed from the **same** spline evaluation as the residual: one evaluation yields both f and the Newton step $\tau \leftarrow \tau + f \cdot (u^0 - \boldsymbol{u}\cdot\hat{\boldsymbol{n}})^{-1}$.
3. **The last evaluation is reused.** The converged residual evaluation carries the full $[x^\mu; u^\mu]$ state and the geometry factors, so the Liénard–Wiechert accumulation costs **zero** extra spline evaluations.

The kernel (`src/gpu/kernel_newton.jl`, sentinel `GPUKernelNewton`) keeps the RK4 kernel's per-pixel window logic verbatim and replaces the march by, per sample: an Euler predictor from the previous sample's converged state ($\tau \leftarrow \tau + \Delta x^0 \cdot (u^0 - \boldsymbol{u}\cdot\hat{\boldsymbol{n}})^{-1}$, free), followed by a **fixed** number `n_iters` of Newton corrections — fixed, so all lanes of a wavefront run in lockstep and no warp divergence is introduced. τ is clamped to the trajectory span; only τ and the Newton factor stay live across samples (one register more than RK4). The per-sample error is set by the last Newton step alone — it does **not** accumulate along the grid, unlike the march.

method	spline evals / sample	error behaviour
RK4, `n_substeps` $= n$	$4n + 1$	accumulates along the march
Newton, `n_iters` $= m$	$m + 1$	per-sample, non-accumulating

Table 1: Cost model. At the regimes measured below, Newton needs $m = 1$ (gentle, $a_0 = 0.1$) to $m = 3$ (forward-Doppler stress) where RK4 needs $n = 1$ to $n = 8$.

3 Validation

V1 — root-level, host. (`scripts/v1_newton_root_check.jl`) Newton-solved τ_r vs a `Vern9` solve of Equation 2 at `reltol = abstol = 1e-12`, on the analytic worldlines of `test/gpu_radiation.jl`. Transverse regime: quadratic convergence to the floor in two corrections (worst pixel: $6.8 \times 10^{-4} \to 2.1 \times 10^{-8} \to 1.2 \times 10^{-11}$ relative $\Delta\tau$ for 0, 1, 2 corrections). Forward-Doppler regime ($v_z = 0.95c$): hard off-axis pixels converge more slowly ($2.5 \times 10^{-1} \to 7.3 \times 10^{-3} \to 3.2 \times 10^{-4} \to 7.0 \times 10^{-7}$) because the per-sample $\Delta\tau_r$ spans a sizable fraction of the trajectory wiggle period, so the predictor lands far from the root.

V2 — kernel-level, CPU backend. The actual kernel run under `KernelAbstractions.CPU()`, accumulated potential vs the adaptive `Vern9` CPU reference (relative L_2):

regime	RK4 n=1	RK4 n=8	Nwt m=1	Nwt m=2	Nwt m=3	Nwt m=4
transverse	$1.94 \cdot 10^{-5}$	$1.95 \cdot 10^{-5}$	$1.95 \cdot 10^{-5}$	$1.95 \cdot 10^{-5}$	$1.95 \cdot 10^{-5}$	$1.95 \cdot 10^{-5}$
fwd. Doppler	$1.05 \cdot 10^{-1}$	$1.56 \cdot 10^{-3}$	$2.74 \cdot 10^{-2}$	$2.08 \cdot 10^{-3}$	$1.49 \cdot 10^{-3}$	$1.49 \cdot 10^{-3}$

Table 2: V2: accumulated-potential error. In the stress regime Newton reaches the RK4 `n_substeps = 8` floor with `n_iters = 3` — at 4 vs 33 spline evaluations per sample.

Both regimes are also encoded as testsets in `test/gpu_radiation.jl` (28 tests pass, including the field path).

V3 — GPU, production-like physics. (`scripts/diag_newton_precision.jl`) $a_0 = 0.1$ circularly-polarized Laguerre–Gauss, 100 electrons, 50^2 pixels, 1000 samples, vs the CPU two-phase path at `reltol = abstol = 1e-12`:

method	rel. error vs tight reference
RK4 `n_substeps = 1`	$9.51 \cdot 10^{-10}$
RK4 `n_substeps = 8`	$5.73 \cdot 10^{-11}$
Newton `n_iters = 1`	$9.36 \cdot 10^{-11}$
Newton `n_iters = 2, 3`	$9.36 \cdot 10^{-11}$ (identical — converged)

Table 3: V3: at $a_0 = 0.1$ a single Newton correction sits at the accuracy floor, an order of magnitude below single-substep RK4.

4 Benchmarks (W7900, ROCm)

Same physics as V3; `sync_per_electron = true`; wall time is min-of-k end-to-end (upload, kernel, download, permute). Speedups are within-process.

potential path	rel. err	time (s)	vs RK4 n=1	vs RK4 n=8	k
$N = 100$, 50^2 px, 1000 samples					
RK4 `n_substeps = 1`	$9.5 \cdot 10^{-10}$	3.004	1.00	5.61	3
RK4 `n_substeps = 8`	anchor	16.866	0.18	1.00	3
Newton `n_iters = 1`	$7.4 \cdot 10^{-11}$	1.665	1.80	10.13	3
Newton `n_iters = 2`	$7.4 \cdot 10^{-11}$	2.134	1.41	7.90	3
$N = 300$, 50^2 px, 1000 samples					
RK4 `n_substeps = 1`	$9.8 \cdot 10^{-10}$	8.713	1.00	5.78	3
RK4 `n_substeps = 8`	anchor	50.401	0.17	1.00	3
Newton `n_iters = 1`	$5.7 \cdot 10^{-11}$	4.430	1.97	11.38	3
$N = 1000$, 200^2 px, 1000 samples					
RK4 `n_substeps = 1`	$9.9 \cdot 10^{-10}$	85.032	1.00	6.15	2
RK4 `n_substeps = 8`	anchor	523.304	0.16	1.00	2
Newton `n_iters = 1`	$5.3 \cdot 10^{-11}$	57.239	1.49	9.14	2
Newton `n_iters = 2`	$5.3 \cdot 10^{-11}$	73.458	1.16	7.12	2

Table 4: Potential path. Relative errors are vs the converged RK4 `n_substeps = 8` anchor of the same rung (absolute accuracy of the anchor pinned by V3).

field path (split mode)	rel. err	time (s)	vs anchor
$N = 300$, 200^2 px, 1000 samples (min of 2)			
RK4 `n_substeps = 8`	anchor	201.821	1.00
RK4 `n_substeps = 1`	$8.2 \cdot 10^{-7}$	48.425	4.17
Newton `n_iters = 1`	$4.9 \cdot 10^{-8}$	50.344	4.01
Newton `n_iters = 2`	$4.9 \cdot 10^{-8}$	58.862	3.43
$N = 300$, 400^2 px, 1000 samples (min of 1, two runs)			
RK4 `n_substeps = 1` (run A)	anchor	158.357	1.00
Newton `n_iters = 2` (run A)	$8.2 \cdot 10^{-7}$	121.939	1.30
RK4 `n_substeps = 1` (run B)	anchor	118.415	1.00
Newton `n_iters = 1` (run B)	$8.2 \cdot 10^{-7}$	103.645	1.14

Table 5: Field path. The 400^2 rows are single-shot timings from two separate processes whose anchors differ by $\sim 30\%$ (GPU clock / thermal state) — only within-run ratios are meaningful. The $8.2 \cdot 10^{-7}$ entries are dominated by the `n_substeps = 1` anchor's own error.

Reading. On the potential path the win tracks the eval-count model (2 evals/sample vs 5) minus fixed overheads: 1.8–2.0× at 50^2, 1.5× at 200^2. On the field path each sample additionally pays an acceleration-spline evaluation, the split Faraday tensors and 12 global accumulations, which dilutes the spline-eval saving: `n_iters = 1` is time-neutral against `n_substeps = 1` at 200^2 (but 17× more accurate) and modestly faster at 400^2. Unlike the earlier FindFirstFunctions/guess-search experiment, there is **no** register-pressure regression — the Newton loop replaces the march instead of adding state to it.

5 Caveats

- **Forward-Doppler pixels need `n_iters = 3`.** Where $u^0 - \boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ is small the predictor is poor (V1/V2). This is the same regime where RK4 needs `n_substeps = 8`, and Newton still gets there $8\times$ cheaper. At $a_0 \lesssim 0.1$ geometries a single correction is at the floor.
- **Cancellation floor.** f subtracts numbers of size $x^0 \approx |\boldsymbol{x}| \approx 3 \cdot 10^9$ a.u. (screen at $Z = 2 \cdot 10^5 \lambda$), so f resolves to $\sim 7 \cdot 10^{-7}$ absolute, i.e. τ to $\sim 5 \cdot 10^{-9}$. Below the observed reference floor, but machine-precision residuals should not be expected, and any f-based tolerance must stay above $\sim 10^{-6}$.
- **Fixed cost ceiling.** Buffer-write traffic is method-independent; on write-bound configurations the eval saving cannot show. The RK4 $n = 1$ vs $n = 8$ spread (5–6×) confirms the measured rungs are eval-bound on the potential path and partially so on the field path.

6 Recommendation and next steps

Adopt the Newton solve as the default retarded-time strategy for the unified kernels: `n_iters = 1` for $a_0 \lesssim 0.1$ radiation campaigns (potential and field), `n_iters = 3` for strongly-relativistic forward-scattering geometries. Staging: `GPUKernelNewton` now lives in the `Experimental` submodule (alongside `GPUKernelTsit5`, re-exported at top level) for production trials behind the existing dispatch seam. Promote to the default once a high-a_0 sweep (where sub-stepping genuinely matters for RK4) and a full-campaign A/B (e.g. a `lowa0_maps` cell) confirm parity.

Code: `src/gpu/kernel_newton.jl` on branch `worktree-newton-lightcone` (commit `ba29f30`). Reproduce: `scripts/v1_newton_root_check.jl`, `scripts/v2_newton_kernel_check.jl` (test env), `scripts/diag_newton_precision.jl`, `scripts/benchmark_newton_gpu.jl`, `scripts/benchmark_newton_field_gpu.jl` (scripts env, W7900). Tests: `julia --project=test test/gpu_radiation.jl`.